```
% Input parameters
xp = [0.75 , -1]; % target point
L1 = 1; % length of arm1
L2 = 1.5; % length of arm2
theta = [2.5 2.7]; % initial angles
tol = 0.01; % tolerance
alpha = 0.1; % learning rate
Niter = 1000; % max iteration number
% Compute tip robot position
function [x,y] = robot_position(theta, L1, L2)
x = L1 * cos(theta(1)) + L2 * cos(theta(2));
y = L1 * sin(theta(1)) + L2 * sin(theta(2));
end
function plot_robot_arm(theta, target, arm_lengths)
l1 = arm_lengths(1);
l2 = arm_lengths(2);
x0 = 0;
y0 = 0;
x1 = l1 * cos(theta(1));
y1 = l1 * sin(theta(1));
x2 = x1 + l2 * cos(theta(2));
y2 = y1 + l2 * sin(theta(2));
% Plot the arm segments
figure;
hold on;
plot([x0, x1], [y0, y1], 'b-', 'DisplayName', 'Segment 1', 'LineWidth',2);
plot([x1, x2], [y1, y2], 'b-', 'DisplayName', 'Segment 2', 'LineWidth',2);
% Plot the target point
plot(target(1), target(2), 'Marker', 'pentagram', 'DisplayName', 'Target',
'MarkerSize', 20,'Color', 'magenta', 'MarkerFaceColor', 'magenta');
plot(x0, y0, 'Marker', 'square', 'MarkerSize', 12, 'Color', 'black',
'MarkerFaceColor', 'black');
plot(x1, y1, 'Marker', 'square', 'MarkerSize', 12, 'Color', 'black',
'MarkerFaceColor', 'black');
plot(x2, y2, 'Marker', 'o', 'MarkerSize', 12, 'Color', 'green', 'MarkerFaceColor',
'green');
% Setting up the plot
plot_max = max([target(1), l1 + l2, target(2)]);
xlim([- plot_max - 0.5, plot_max + 0.5]);
ylim([- plot_max - 0.5, plot_max + 0.5]);
axis equal;
grid on;
title('Robotic Arm Configuration');
xlabel('X');
ylabel('Y');
% Show the plot
hold off;
```

```
end
% Plot robot arms
[x,y] = robot_position(theta, L1, L2);
fprintf('%f4.2 %f4.2 \n',[x,y]);
plot_robot_arm(theta, xp, [L1, L2]);
% Evaluation of J
function Jval = J(theta, xp, L1, L2)
[x, y] = robot_position(theta, L1, L2);
Jval = (x-xp(1))^2 + (y-xp(2))^2;
end
Jval = J(theta, xp, L1, L2);
fprintf('%f4.2 \n',Jval);
% Evaluation of grad(J)
function grad = grad_J(theta, xp, L1, L2)
[x, y] = robot_position(theta,L1,L2);
dx_dt1 = - L1 * sin(theta(1));
dx_dt2 = - L2 * sin(theta(2));
dy_dt1 = L1 * cos(theta(1));
dy_dt2 = L2 * cos(theta(2));
dJ_dx = 2*(x - xp(1));
dJ_dy = 2*(y - xp(2));
DJ_dt1 = dJ_dx*dx_dt1 + dJ_dy*dy_dt1;
DJ_dt2 = dJ_dx*dx_dt2 + dJ_dy*dy_dt2;
grad = [DJ_dt1, DJ_dt2];
end
% Gradient Descent Method
i = 1;
while i <= Niter
% Gradient of the objective function
grad = grad_J(theta, xp, L1, L2);
% Update theta with gradient
theta = theta - alpha * grad;
% Compute the current distance
Jval = J(theta, xp, L1, L2);
% Check for convergence
if Jval < tol
fprintf('Converged after %d iterations.\n', i)
break
end
i = i + 1;
end
fprintf('Final angle combination: %f4.2 %f4.2 \n', theta)
fprintf('Final distance: %f4.2 \n', Jval)
plot_robot_arm(theta, xp, [L1, L2]);
```